MicrojuegoAsteroids_GDD

Jaime Calero Toledano 50379992X

1. Escena y configuración 2D

El proyecto se organiza alrededor de MainScene.unity, una escena 2D con cámara ortográfica. La escala de los objetos no depende de la profundidad y el fondo espacial queda detrás de todas las entidades de juego.

Parámetro	Configuración
Cámara	Orthographic = true; Size = 5.0; posición (0, 0, -10).
Área visible	Altura de 10 unidades; en 16:9, ancho aproximado 17.77 (X = -8.88 a +8.88).
Clipping planes	Near = 0.3; Far = 1000.
Fondo	RGBA(13, 13, 20, 255). Background: SpriteRenderer con Space.jpg en (0, 0, 5), sortingOrder = -10.
Ejes	X horizontal; Y vertical; Z perpendicular a la pantalla. Las entidades interactivas usan Z = 0.

2. Assets y GameObjects

Objeto	Componentes / asset / función
Player	Transform, SpriteRenderer, Rigidbody, SphereCollider y Player.cs. Ship.png (529 x 577). Tag Player / Default.
Gun	Transform hijo de Player en X = +0.6. Origen de los disparos.
BulletPrefab	SpriteRenderer, SphereCollider Trigger y Bullet.cs. Bullet.png (4 x 4). Tag Bullet.
AsteroidPrefab	SpriteRenderer, SphereCollider Trigger y Asteroid.cs. Meteor.png (1000 x 1000). Tag Enemy.
MiniAsteroidPrefab	Meteor.png a escala 0.5x, SphereCollider y Asteroid.cs. Fragmento de una colisión.
EnemySpawner / ObjectPoolManager	Scripts EnemySpawner.cs y ObjectPool.cs. Generación y reciclaje de entidades.
Canvas / EventSystem	Canvas, CanvasScaler, GraphicRaycaster y UIManager.cs; EventSystem y StandaloneInputModule. HUD y entrada de UI.

La nave apunta naturalmente a Vector3.right. Todos los objetos de juego usan la capa Default salvo la interfaz, que usa UI.

3. Mecánicas de juegoç

3.1 Movimiento de la nave

La nave se mueve con Rigidbody.AddForce en la dirección transform.right. El amortiguamiento lineal (linearDamping / drag = 1.2) mantiene la inercia al soltar el acelerador. El giro se hace directamente con transform.Rotate(Vector3.forward, -rotation), por lo que responde de forma inmediata.

- Envoltorio toroidal: si |X| > 9.2 o |Y| > 5.3, la nave aparece en el extremo contrario.
- La física controla el desplazamiento; el script controla la orientación.

3.2 Disparo

- Se dispara con barra espaciadora o Fire1. El proyectil nace en Gun y hereda la orientación de la nave.
- Velocidad lineal: 14 m/s, usando targetVector * speed * Time.deltaTime.
- Vida máxima de la bala: 3 s. Después se desactiva y vuelve al pool si no ha impactado.

3.3 Generación de enemigos

- Los meteoritos aparecen en Y = +6.2 y en una X aleatoria entre -8.0 y +8.0.
- La tasa inicial es de 30 asteroides por minuto y aumenta en 1.5 por minuto con cada generación.
- Intervalo de generación: 60 / spawnRatePerMinute.

3.4 Fragmentación angular (Iteración 2)

Según la especificación de clase, cada colisión asteroide-bala produce 2 mini-asteroides. El vector de la bala es la bisectriz de las dos trayectorias. Con alpha = 35 grados:

theta_b = atan2(v_y, v_x)
theta_1 = theta_b + alpha theta_2 = theta_b - alpha
u_1 = (cos(theta_1), sin(theta_1)) u_2 = (cos(theta_2), sin(theta_2))

Los fragmentos tienen escala 0.5x y velocidad relativa 3.2 m/s. Un segundo impacto los destruye de forma permanente y suma puntos extra.

4. Interfaz y controles

Acción	Primaria	Alternativa / interfaz
Rotar izquierda	Flecha izquierda	A
Rotar derecha	Flecha derecha	D
Empuje	Flecha arriba	W
Disparo	Espacio	Ctrl / clic izquierdo
Pausa / reanudar	Escape	Botón Pausa (Esc) en HUD

5. Arquitectura software

La arquitectura separa el control del jugador, el spawn, la interfaz, el combate y la gestión de memoria. GameManager coordina el estado general del juego.

Script	Responsabilidad
Player.cs	Rigidbody, ejes Rotate y Thrust, entradas estándar, giro, disparos, límites y variable Player.SCORE.
Bullet.cs	Movimiento, temporizador, OnTriggerEnter(Collider), detección de Enemy, +10 puntos y refresco de UI.
Asteroid.cs	Movimiento descendente, rotación, TakeHit(Vector3 bulletDirection), fragmentación, colisión con Player y reinicio.
EnemySpawner.cs	Spawn temporal acelerado e invocación de objetos desde la piscina.
ObjectPool.cs	Singleton con colas y métodos GetBullet, GetAsteroid, GetMiniAsteroid y sus retornos.
GameManager.cs	Estados Jugando, Pausado y Game Over; Time.timeScale; tecla Escape y botones.
UIManager.cs	Texto de puntuación y visibilidad de los paneles de pausa y derrota.

Relación principal: GameManager coordina Player, EnemySpawner y UIManager. Player se relaciona con Bullet; Bullet y Asteroid resuelven los impactos; las entidades se apoyan en ObjectPool.

6. Pausa y reinicio

La pausa se activa con Escape o con el botón superior derecho. Al pausar, Time.timeScale = 0f y se congelan las físicas, las balas, los meteoritos y el contador del spawner.

Opción	Resultado
Reanudar	Time.timeScale = 1f y se oculta el panel.
Reiniciar	Marcador a 0, tiempo normal y recarga de escena mediante SceneManager.LoadScene().
Salir	Cierra la aplicación o termina el modo Play en el editor de Unity.

7. Flujo de una partida

- El jugador rota y acelera dentro de una pantalla toroidal.
- EnemySpawner crea meteoritos progresivamente desde la parte superior.
- El jugador dispara; Bullet detecta Enemy, actualiza el marcador y devuelve entidades a la piscina.
- Asteroid gestiona el impacto: el primario puede crear dos fragmentos con separación angular.
- Una colisión con Player notifica la muerte de la nave y reinicia la escena.

8. Object Pooling y rendimiento

Instanciar y destruir continuamente balas, meteoritos y fragmentos provoca asignaciones efímeras en el heap y puede generar pausas de Garbage Collection. Por eso ObjectPool.cs prepara los objetos en Awake y los reutiliza durante la partida.

Pool	Reserva inicial	Funcionamiento
Balas	25	GetBullet: extraer, activar y posicionar. ReturnBullet: desactivar y reencolar.
Asteroides	15	GetAsteroid y ReturnAsteroid siguiendo el mismo ciclo.
Mini-asteroides	30	GetMiniAsteroid y ReturnMiniAsteroid.

Ciclo de reutilización

1) Queue.Dequeue extrae un objeto disponible. 2) Se activan sus componentes y se coloca en escena. 3) Tras su ciclo de vida o un impacto se hace SetActive(false). 4) Se reparenta al contenedor inactivo y se ejecuta Queue.Enqueue. El objetivo de diseño es evitar Garbage Collection durante el juego activo (aprox. 0 bytes/frame).

9. Resumen de implementación

Área	Criterio
Escena	MainScene carga con cámara ortográfica, fondo en sortingOrder -10 y entidades en Z = 0.
Movimiento	Rigidbody, inercia, giro inmediato y wrapping en los cuatro bordes.
Disparo	Bala en Gun, 14 m/s, vida de 3 s y retorno al pool.
Spawn	Y = 6.2, X aleatoria y dificultad progresiva.
Combate	+10 por impacto; fragmentación de primarios con bisectriz a 35 grados.
Estados	Jugando, Pausado y Game Over; Escape, botones y reinicio funcionales.
Interfaz	Score, botón de pausa, paneles modales y EventSystem.